

# 校园互助服务平台 - 详细设计文档（整合版）

阶段：Phase 3 详细设计

版本：1.0

最后更新：2026-05-17

## 1. 引言

本文档整合了校园互助服务平台详细设计阶段的所有产出，包括类图设计（含 SOLID 检查）、API 规范、数据库设计（ER 图 + 建表 SQL）以及 AI 协作反思日志。各详细内容请参见对应的独立交付物文件。

## 2. 类图设计

类图设计遵循 SOLID 原则，涵盖用户认证、互助订单（跑腿/二手/组队/失物招领）、聊天会话、评价等核心模块。

- 类图文件：**类图.md（包含完整的类定义、属性、方法及 Mermaid 关系图）
- SOLID 检查清单：**SOLID检查清单.md（记录 AI 原始设计的违规项及修正方案）

### 2.1 核心设计摘要

- 通过 BaseOrder 抽象基类与 PaidOrder 子类，实现了带积分订单与纯信息订单的分离，满足里氏替换原则。
- User 类剥离了隐私配置到 PrivacyProfile，符合单一职责原则。
- 订单类型（跑腿、二手、组队）继承自 PaidOrder，失物招领继承自 BaseOrder，支持开闭原则扩展新业务。

详细类图及关系说明请见 类图.md，SOLID 检查详情请见 SOLID检查清单.md。

## 3. API 规范设计

采用 RESTful 风格，覆盖核心业务接口，使用 JWT 进行认证。

- API 规范文件：**API规范文档.yaml（OpenAPI 3.0 格式）

### 3.1 接口列表摘要

| 模块 | 接口                        | 方法   | 说明                  |
|----|---------------------------|------|---------------------|
| 用户 | /user/register            | POST | 学号注册                |
| 用户 | /user/login               | POST | 登录获取 token          |
| 需求 | /requirements             | POST | 发布需求（跑腿/二手/组队/失物招领） |
| 需求 | /requirements             | GET  | 浏览需求列表（筛选+分页）       |
| 需求 | /requirements/{id}/accept | POST | 接单                  |

| 模块 | 接口                            | 方法   | 说明         |
|----|-------------------------------|------|------------|
| 订单 | /orders/{orderId}             | GET  | 查看订单详情     |
| 订单 | /orders/{orderId}/evaluations | POST | 提交评价（1~5星） |

3.2 通用响应格式

```
{
  "code": 200,
  "msg": "操作成功",
  "data": {}
}
```

完整的请求/响应示例、错误码定义请参见 [API规范文档.yaml](#)。

4. 数据库设计

数据库采用 MySQL（utf8mb4 字符集），支持校园互助平台的所有实体存储。

- **ER 图文件：**[ER图.md](#)（Mermaid 格式，含实体关系及属性）
- **建表 SQL 文件：**[建表SQL代码.txt](#)（可直接执行的 SQL 语句）

4.1 核心表摘要

| 表名                                      | 说明                  | 关键索引                                             |
|-----------------------------------------|---------------------|--------------------------------------------------|
| user                                    | 用户信息                | student_id 唯一索引                                  |
| privacy_profile                         | 隐私配置                | user_id 唯一索引                                     |
| user_account                            | 积分与信誉账户             | user_id 唯一索引                                     |
| base_order                              | 订单基表                | publisher_id, acceptor_id, (status, create_time) |
| paid_order                              | 积分订单（继承 base_order） | 主键外键                                             |
| errand_order / trade_order / team_order | 具体业务订单              | 主键外键                                             |
| lost_found_order                        | 失物招领（继承 base_order） | 主键外键                                             |
| chat_session / chat_message             | 聊天会话与消息             | session_id, (session_id, send_time)              |
| evaluation                              | 评价                  | order_id, target_user_id                         |

4.2 索引设计要点

- `user.student_id`: 学号登录高频查询，唯一索引保证不重复。
- `base_order.status, create_time`: 首页需求列表排序查询，复合索引提升性能。
- `chat_message.session_id, send_time`: 会话历史记录按时间分页查询。

完整建表语句及外键约束请参见 [建表SQL代码.txt](#)，ER 图见 [ER图.md](#)。

## 5. 设计模式应用

在详细设计中，针对系统中的多态性业务流转和跨模块状态联动场景，应用了以下两种设计模式，以提高系统的可扩展性和解耦度。

### 5.1 策略模式（Strategy Pattern）—— 订单匹配与 AI 推荐机制

#### 5.1.1 业务背景

平台中的不同订单类型具有完全不同的匹配与推荐算法：

- **跑腿订单** (`ErrandOrder`)：基于距离（取件地与送达地）的最优路线物理匹配，或基于抢单机制。
- **失物招领订单** (`LostFoundOrder`)：需要 AI 辅助匹配（文本相似度、嵌入向量等），在“寻物”与“招领”之间做双向语义匹配。
- **组队订单** (`TeamOrder`)：基于兴趣标签、课程编号、成员上限与当前人数进行重合度筛选。

#### 5.1.2 模式应用方案

定义统一的匹配策略接口 `OrderMatchingStrategy`：

```
public interface OrderMatchingStrategy {
    List<Order> match(BaseOrder order);
}
```

针对不同订单类型分别实现具体策略：

- `DistanceMatchingStrategy`：用于跑腿订单的距离匹配。
- `AIMatchingStrategy`：用于失物招领的语义匹配。
- `TagMatchingStrategy`：用于组队订单的标签匹配。

在业务层引入上下文（Context），根据订单类型动态注入对应的策略。

#### 5.1.3 使用理由

- **符合开闭原则**：未来增加新互助类型（如“学习互助导师匹配”），只需新增 `MentorMatchingStrategy` 并注册，无需修改现有匹配逻辑。
- **隔离算法复杂性**：AI 匹配、地理位置计算等复杂逻辑封装在独立策略类中，保持订单核心业务（发布、取消）的纯净。

#### 5.1.4 不使用该模式的后果

若不使用策略模式，Service 层将出现大量 `if-else` 或 `switch-case` 类型判断：

```
if (order instanceof ErrandOrder) {  
    // 距离匹配逻辑（几百行）  
} else if (order instanceof LostFoundOrder) {  
    // AI 匹配逻辑（几百行）  
} else if (order instanceof TeamOrder) {  
    // 标签匹配逻辑  
}  
}
```

- 代码臃肿难维护，一个方法可能达数千行。
- 扩展性极差：新增订单类型必须修改核心匹配方法，极易引入 Bug。

## 5.2 观察者模式（Observer Pattern）—— 订单状态流转与跨模块消息通知

### 5.2.1 业务背景

当一个积分互助订单状态发生改变时（例如接单者点击“完成任务”进入待确认，或发布者点击“确认完成”），系统需要同时触发多个不同模块的连带反应：

1. **积分与账户模块**：将 `frozenPoints` 解冻，并转入接单者的 `availablePoints`。
2. **社交与反馈模块**：`ChatSession` 自动发送系统消息（如“交易已完成，快去评价吧！”），并推送评价邀请。
3. **通知系统**：触发微信模板消息或应用内 Push，通知双方状态更新。

### 5.2.2 模式应用方案

将 `PaidOrder`（或订单服务）作为**观察目标（Subject）**，将积分结算服务、聊天服务、通知服务等作为**观察者（Observer）**。

```
// 观察者接口  
public interface OrderStatusObserver {  
    void onOrderCompleted(Order order);  
}  
  
// 具体观察者  
public class AccountObserver implements OrderStatusObserver { ... }  
public class ChatObserver implements OrderStatusObserver { ... }  
public class NotificationObserver implements OrderStatusObserver { ... }
```

当订单状态变更为“已完成”时，目标对象通知所有已注册的观察者依次执行各自的业务逻辑。

### 5.2.3 使用理由

- **模块间彻底解耦**：订单模块在状态变更后，不需要知道后续有多少个业务要执行，只需发布事件。符合架构设计中的低耦合要求。

- **动态可配置**：运营需求（如“首单奖励活动”）只需动态注册一个新观察者，订单模块代码无需改动。

5.2.4 不使用该模式的后果

若不使用观察者模式，订单状态流转方法中将充斥着对其他服务的硬编码调用：

```
public void completeOrder(Long orderId) {
    orderMapper.updateStatus(orderId, Status.COMPLETED);
    accountService.releasePoints(...);    // 强耦合积分模块
    chatService.sendSystemMessage(...);  // 强耦合聊天模块
    pushService.sendPush(...);           // 强耦合推送模块
}
```

- **违反单一职责原则**：订单服务被迫管理积分、聊天、推送等外部领域逻辑。
- **性能脆弱**：若推送或积分写入因网络延迟卡死，整个订单完成操作将失败，核心链路极其脆弱。
- **团队协作冲突**：多人同时修改同一个 Service 方法，频繁产生代码冲突。

6. AI 协作反思日志

本阶段使用 AI 辅助生成了类图初稿、API 文档初稿、建表 SQL 初稿，并进行了 SOLID 原则审查。

- **反思日志文件**：[AI协作反思日志#3.md](#)（按模板填写）

6.1 核心结论摘要

- **AI 助力**：生成建表 SQL 效率极高（节省约 70% 时间），类图结构基本合理。
- **AI 误导**：初始类图违反里氏替换原则（`LostFoundOrder` 继承了含积分逻辑的父类），以及单一职责原则（`User` 类混杂认证与隐私逻辑），需要人工修正。
- **Prompt 改进**：后续要求 AI 在生成类图时明确标注每个类的职责，并自行做一次 SOLID 自查。

详细反思内容（包括 AI 使用清单、误导分析、工程判断等）请参见 [AI协作反思日志#3.md](#)。

7. 交付物索引

| 序号 | 交付物          | 文件位置                          |
|----|--------------|-------------------------------|
| 1  | 类图           | <a href="#">类图.md</a>         |
| 2  | SOLID 检查清单   | <a href="#">SOLID检查清单.md</a>  |
| 3  | API 规范文档     | <a href="#">API规范文档.yaml</a>  |
| 4  | ER 图         | <a href="#">ER图.md</a>        |
| 5  | 建表 SQL       | <a href="#">建表SQL.sql</a>     |
| 6  | AI 协作反思日志 #3 | <a href="#">AI协作反思日志#3.md</a> |
| 7  | 详细设计文档（整合版）  | 本文档                           |